Renox: Dokumen Arsitektur Platform

Oct 26, 2026 · @Achmad Arif Rachim

1. Ringkasan

Renox adalah platform backend dan web dalam satu binary Rust. Binary ini di-build sekali, lalu menjalankan aplikasi yang ditulis dalam TypeScript, template HTML, dan SQL tanpa perlu rebuild. Platform menyediakan static hosting, server side rendering (SSR), function/API, database Postgres, dan sistem modul yang bisa disusun seperti Lego.

Node.js tidak dipakai sama sekali. Kode TypeScript dijalankan oleh engine JavaScript yang di-embed sebagai library di dalam proses Rust.

Tujuan

1. **Development cepat.** Mengubah halaman, logic, atau modul cukup dengan menyimpan file. Hot reload di bawah 1 detik, tanpa compile Rust dan tanpa `npm install`.
2. **Runtime kencang.** Static file, routing, render HTML, dan akses database dikerjakan oleh Rust. JavaScript hanya menjalankan business logic.
3. **Hemat memory.** Satu proses untuk seluruh aplikasi. Runtime JS dibuat sekali per worker thread, dan modul hanya menambah context ringan.
4. **Modular.** Fitur seperti auth, 2FA, billing, dan admin bisa dipasang, di-upgrade, dan di-override per modul.
5. **Satu binary.** Deploy cukup dengan satu file executable dan satu database Postgres.

Non-goals (versi awal)

- Menjalankan Next.js atau framework yang bergantung penuh pada Node API.
- Menjadi SPA framework. Interaktivitas client mengandalkan htmx, Alpine.js, dan islands.
- Multi-region / edge deployment. Target awal adalah satu region, satu cluster Postgres.
- Marketplace modul publik. Distribusi awal lewat path lokal dan git.

2. Keputusan arsitektur

Setiap keputusan di bawah diambil untuk memenuhi tiga tujuan sekaligus: development cepat, runtime kencang, dan memory hemat. Alternatif yang ditolak dicatat supaya keputusan bisa ditinjau ulang kalau kondisinya berubah.

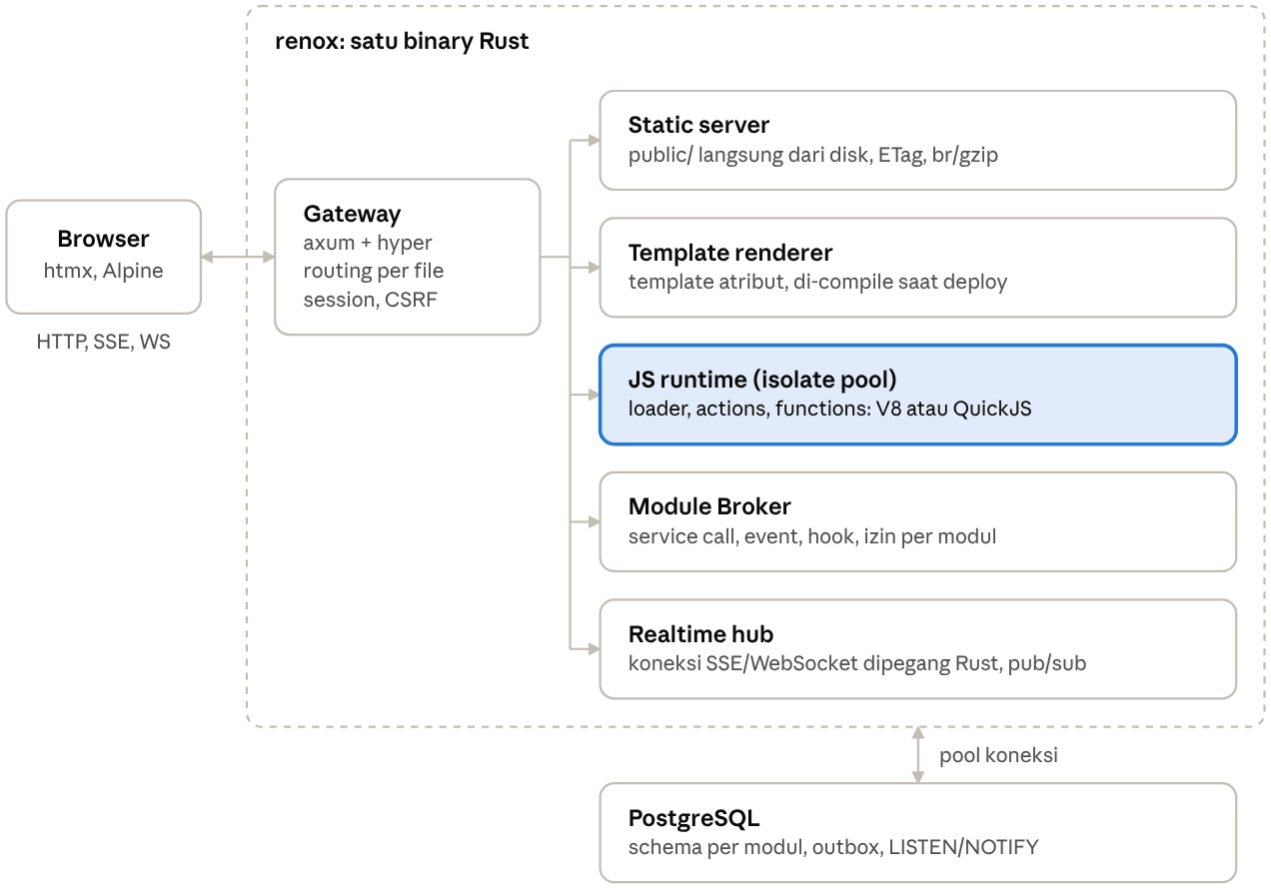
#	Keputusan	Alasan	Alternatif yang ditolak
D1	Platform ditulis dengan Rust (tokio + hyper/axum)	Performa dan memory mendekati C, aman dari kelas bug memory, ekosistem async matang	Go (GC pause, memory lebih besar), Node (boros memory per proses)
D2	User code ditulis dalam TypeScript, dijalankan oleh engine JS yang di-embed	Developer tidak perlu rebuild Rust. TS punya ekosistem dan tooling yang luas	Rust untuk user code (harus compile ulang), bahasa script baru seperti Rhai (kurva belajar)
D3	Tanpa Node.js. Engine JS adalah library di dalam binary	Satu proses, satu binary, tidak ada IPC antar proses, footprint kecil	Node/Bun sebagai sidecar (proses tambahan, memory dobel, latency IPC)
D4	Engine default V8 via `deno_core`, dibungkus abstraksi `JsEngine`	JIT tercepat, kompatibilitas terbaik, Web API dan Node polyfill dari ekosistem Deno bisa dipakai ulang	QuickJS sebagai default (lebih hemat, tapi 10–50x lebih lambat untuk komputasi berat). Tetap dipertahankan sebagai opsi
D5	Render HTML dilakukan Rust, JS hanya mengembalikan data	Render adalah bagian SSR yang paling berat. Di Rust lebih cepat dan tidak membebani heap JS	SSR penuh di JS dengan JSX (lebih lambat dan boros memory). Tetap tersedia sebagai opsi kedua
D6	Template engine berbasis atribut HTML (`r-if`, `r-for`, `{{ }}`)	Template tetap HTML valid, menyatu dengan htmx dan Alpine, dan tooling editor tetap bekerja	Jinja/minijinja (sintaks blok tidak disukai), Handlebars
D7	Interaktivitas lewat fragment HTML + htmx + idiomorph + View Transitions	Partial update tanpa SPA, state DOM dan animasi terjaga, JS client kecil	React/SPA (bundle besar, state ganda client/server)
D8	Convention over configuration untuk routing	Tanpa boilerplate router. Struktur folder sama dengan struktur URL	Router manual di config (lambat ditulis, mudah tidak sinkron)

#	Keputusan	Alasan	Alternatif yang ditolak
D9	Modul = mini app dengan struktur folder yang sama	Satu konsep untuk dipelajari. App dan modul diproses oleh loader yang sama	Plugin API khusus (konsep kedua yang harus dipelajari)
D10	Komunikasi antar modul in-process, diperantarai Rust	Latency mikrodetik, transaksi DB lintas modul bisa dilakukan, permission ditegakkan di satu tempat	Microservices / HTTP antar modul (latency, tanpa transaksi, operasional berat)
D11	Satu isolate per worker thread, satu context per modul	Biaya runtime dibayar sekali. Modul hanya menambah ratusan KB	Isolate per modul atau per request (memory dan cold start berlipat)
D12	PostgreSQL dengan satu schema per modul	Isolasi data alami, migration per modul, transaksi ACID lintas modul	Database per modul (tanpa transaksi lintas modul), SQLite (konkurensi tulis terbatas)
D13	Durable event memakai transactional outbox di Postgres	Event dan data tersimpan dalam satu transaksi, tanpa message broker tambahan	Kafka/NATS (komponen operasional tambahan untuk skala yang belum dibutuhkan)
D14	Otorisasi user: RBAC + ABAC + relasi dengan Cedar (cedar-policy), ditambah RLS Postgres	Native Rust, policy di-load saat runtime tanpa rebuild, bisa divalidasi sebelum deploy. RLS menjamin isolasi tenant walaupun kode modul lupa memfilter	OPA/Rego (sidecar Go), Casbin (ABAC kurang ekspresif), predicate TypeScript (tidak bisa dianalisis)

3. Arsitektur sistem

Seluruh platform berjalan sebagai satu proses Rust di atas runtime async tokio. Jalur yang paling sering dipakai (static file, routing, render HTML, query DB, koneksi realtime) sepenuhnya dikerjakan Rust. Engine JS hanya dipanggil untuk loader, action, function, dan handler event.

Satu proses Rust melayani semua jalur, JS hanya menjalankan logic



arsitektur renox · 1 proses, 6 komponen, 1 database

Gateway meneruskan setiap request ke komponen yang sesuai. Hanya JS runtime (yang diberi warna) yang menjalankan kode user.

Komponen	Tanggung jawab	Crate utama
Gateway	HTTP/1.1, HTTP/2, TLS, routing, session, CSRF, kompresi	`hyper`, `axum`, `tower-http`, `rustls`
Static server	Serve `public/` dengan ETag, range request, dan pre-compressed asset	`tower-http::services::ServeDir`
Template renderer	Compile dan render template atribut, fragment, komponen	`html5ever` + compiler internal
JS runtime	Worker pool, isolate, context per modul, ops ke Rust	`deno_core` (V8) atau `rquickjs`, `deno_ast` (swc)
Module Broker	Registry modul, service call, event bus, hook, izin	internal

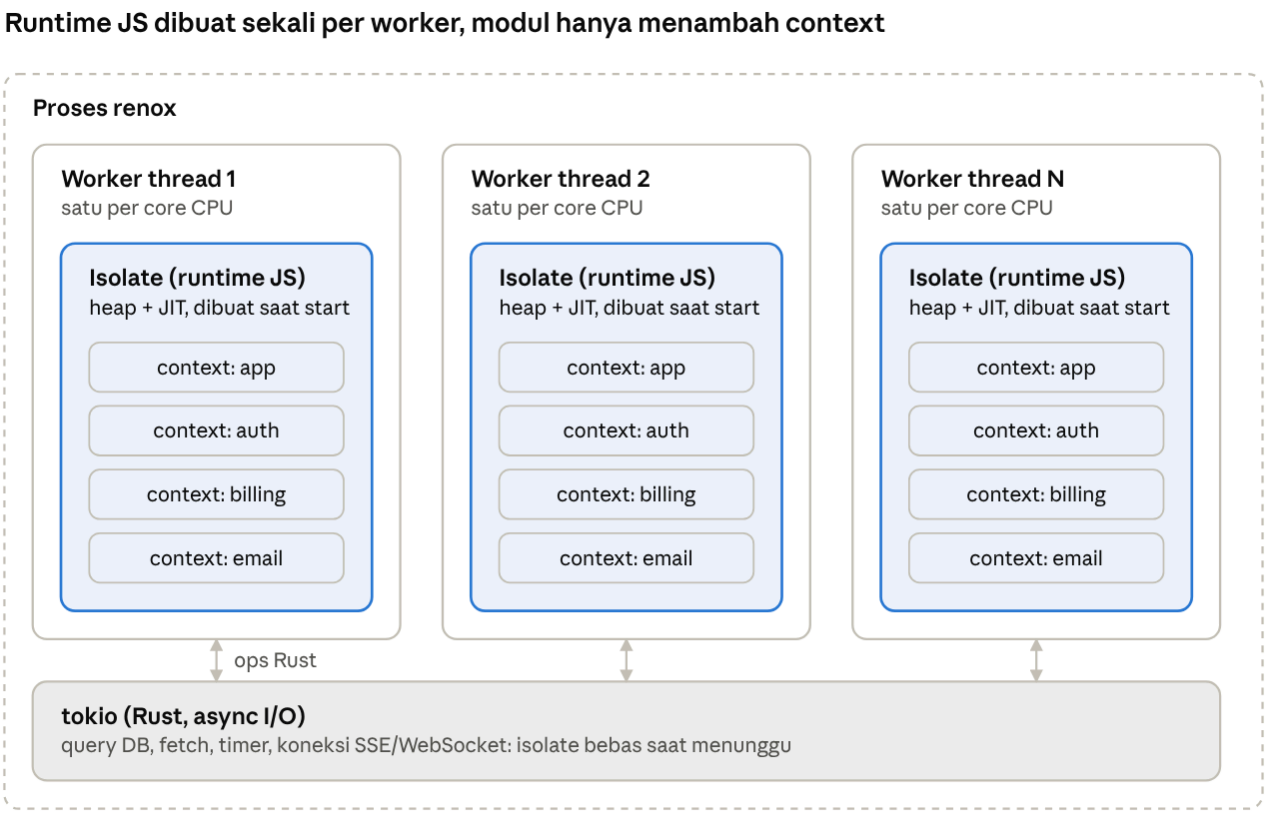
Komponen	Tanggung jawab	Crate utama
Realtime hub	SSE, WebSocket, pub/sub topic, fan-out antar instance	`axum::extract::ws`, `tokio::sync::broadcast`
Data layer	Connection pool, transaksi per request, migration, outbox	`sqlx` atau `tokio-postgres`, `deadpool`

Jalur request SSR:

1. Gateway mencocokkan URL dengan route table yang dibangun dari file.
2. Kalau ada loader, JS runtime menjalankan `load()` di context modul pemilik halaman. Query DB dijalankan oleh op Rust.
3. Template renderer menerima data dalam bentuk nilai serde, lalu merender halaman penuh atau hanya fragment yang diminta.
4. Response di-stream ke client.

4. Model eksekusi JavaScript

Runtime JS (isolate) dibuat sekali per worker thread saat server start, lalu dipakai ulang untuk semua request dan semua modul. Setiap modul hanya mendapat context di dalam isolate tersebut. Tidak ada runtime baru per request, per modul, atau per panggilan antar modul.



model eksekusi · worker, isolate, context per modul

Setiap worker punya isolate sendiri berisi context untuk semua modul. Operasi I/O diserahkan ke tokio, sehingga isolate bisa melayani request lain sambil menunggu.

Biaya per lapisan

Angka di bawah adalah perkiraan awal untuk V8 dan harus divalidasi lewat benchmark di fase F1.

Lapisan	Kapan dibuat	Perkiraan biaya
Isolate	Sekali per worker, saat startup	Sekitar 3–10 MB
Context per modul	Sekali per modul per isolate, dari snapshot	Sekitar ratusan KB
Request	Setiap request	Pemanggilan fungsi, tanpa alokasi runtime
Panggilan antar modul	Setiap service call	Pindah context di thread yang sama, plus serialisasi

Perkiraan memory JS total: `jumlah_worker × (isolate + jumlah_modul × context + heap kerja)`. Untuk 8 worker dan 10 modul, hasilnya berada di kisaran puluhan MB. Sebagai pembanding, satu proses Node.js per layanan biasanya memakai beberapa puluh MB per proses.

Konkurensi

- **Event loop per isolate.** Saat handler menjalankan `await db.query(...)`, op Rust mengerjakan query di tokio dan isolate langsung beralih ke request lain. Satu worker bisa melayani ratusan request yang sedang menunggu I/O.
- **Penjadwalan request ke worker:** least-loaded berdasarkan jumlah request aktif. Isolate V8 terikat ke satu thread, jadi request tidak berpindah worker di tengah eksekusi.
- **Kode CPU-bound** memblokir worker tersebut sampai selesai. Batas CPU time per request (bagian 10) mencegah satu handler menahan worker terlalu lama.

Startup cepat dengan snapshot

Saat deploy, Rust menjalankan inisialisasi setiap modul (top-level code, import) sekali, lalu menyimpan hasilnya sebagai V8 startup snapshot. Worker baru dan context baru dibuat dari snapshot ini, bukan dengan menjalankan ulang kode. Targetnya: membuat context dalam hitungan milidetik.

Pilihan engine

Engine dibungkus trait `JsEngine` (eksekusi modul, panggil fungsi, registrasi op, konversi nilai serde), sehingga engine bisa diganti tanpa mengubah komponen lain.

Engine	Memory	Kecepatan	Kompatibilitas	Posisi
V8 (`deno_core`)	Sekitar 3–10 MB per isolate	Tercepat (JIT)	Terbaik, Web API + Node polyfill dari Deno	Default
QuickJS (`rquickjs`)	Di bawah 1 MB per runtime	Jauh lebih lambat untuk komputasi berat (tanpa JIT)	Cukup untuk business logic, banyak library npm bermasalah	Opsi untuk perangkat kecil
Boa (`boa_engine`)	Kecil	Lebih lambat, belum sematang V8	Terbatas	Dipantau, belum dipakai

Engine	Memory	Kecepatan	Kompatibilitas	Posisi
Wasm (`wasmtime`)	Sangat kecil	Mendekati native	Harus di-compile dulu (Rust, Go, AssemblyScript)	Untuk function berat, tahap lanjut

Alasan V8 sebagai default: SSR dan business logic sering memakai library dari ekosistem JS, dan V8 memberi kompatibilitas serta performa terbaik. QuickJS tetap dipertahankan lewat trait yang sama, untuk target yang memorinya sangat terbatas.

5. Struktur aplikasi dan routing

URL dan perilaku ditentukan oleh lokasi dan nama file (convention over configuration). Konfigurasi hanya dipakai sebagai jalan keluar untuk kasus yang tidak cocok dengan konvensi.

```
my-app/
├── renox.toml              config app + daftar modul
├── public/                 static hosting, di-serve langsung oleh Rust
├── pages/                  SSR
│   ├── _layouts/main.html  layout (file berawalan _ bukan route)
│   ├── _404.html
│   ├── index.html          /
│   └── products/
│       ├── index.html      /products
│       ├── index.ts
│       ├── [id].html       /products/:id
│       └── [id].ts
├── partials/               komponen template (<x-nama>)
├── functions/              API, contoh: functions/api/checkout.ts ->
/api/checkout
├── migrations/             SQL milik app
└── overrides/<modul>/      menimpa template modul tanpa fork
```

Aturan routing

Pola	Contoh file	URL
Statis	`pages/about.html`	`/about`
Index folder	`pages/products/index.html`	`/products`

Pola	Contoh file	URL
Parameter	`pages/products/[id].html`	`/products/42` → `params.id = "42"`
Catch-all	`pages/docs/[...slug].html`	`/docs/a/b/c` → `params.slug = ["a","b","c"]`
Bukan route	`pages/_layouts/*`, `_404.html`	tidak bisa diakses langsung
API	`functions/api/orders.ts` dengan export `GET` / `POST`	`/api/orders`

Kalau ada dua route yang bentrok, prioritasnya: statis → parameter → catch-all. Bentrokan yang ambigu (misalnya dua modul me-mount path yang sama) ditolak saat startup, bukan saat request.

Pasangan `.ts` dan `.html` tidak wajib

File yang ada	Perilaku
`.html` saja	Template dirender tanpa data dari loader
`.ts` saja	`load()` wajib mengembalikan `Response` (redirect, JSON, HTML custom)
`.ts` + `.html`	`load()` mengembalikan object → dirender oleh template

Aturan yang sama berlaku di dalam `load()`: mengembalikan `Response` berarti template dilewati, mengembalikan object berarti object itu menjadi data template.

```
// pages/products/[id].ts
import type { LoaderContext } from "renox";

export async function load({ params, db }: LoaderContext) {
  const product = await db.queryOne("select * from products where id = $1",
[params.id]);
  if (!product) return new Response(null, { status: 404 });
  return { product, inStock: product.stock > 0 };
}

export const actions = {
  async addToCart({ form, db, session }) { /* ... */ },
};
```

Alasan dua file, bukan single-file component: editor langsung memberi syntax highlight dan type checking TypeScript tanpa plugin, dan parser di Rust jadi lebih sederhana. Format satu file (`<script server>`) bisa ditambahkan nanti sebagai syntactic sugar yang dipecah saat deploy.

Jalan keluar lewat konfigurasi

```
[[routes]]
path = "/p/:id"
page = "products/[id]"     # alias URL lama
```

6. Template engine

Template adalah HTML valid, dan logikanya ditulis sebagai atribut. Template di-compile sekali saat deploy menjadi instruksi render, lalu dieksekusi oleh Rust di setiap request. Semua atribut `r-*` dan `:*` dihapus dari output, sedangkan atribut `hx-*` (htmx) dan `x-*` (Alpine) dibiarkan.

```
<r-layout use="main">
  <ul id="product-list" r-fragment="product_list">
    <li r-for="p in products" class="card" :class="{ 'card--empty': p.stock
== 0 }">
      <a href="/products/{{ p.id }}">{{ p.name }}</a>
      <span r-if="p.stock == 0">Stok habis</span>
      <span r-else>Rp {{ p.price | rupiah }}</span>
      <x-add-button :product="p" />
    </li>
    <li r-empty="products">Produk tidak ditemukan</li>
  </ul>
</r-layout>
```

Sintaks

Kebutuhan	Sintaks
Interpolasi (auto-escape)	`{{ user.name }}`
Filter	`{{ p.price | rupiah }}`
HTML mentah	`r-html="post.body"` (harus eksplisit)
Kondisi	`r-if`, `r-else-if`, `r-else`
Loop	`r-for="p in items"`, `r-for="(p, i) in items"`

Kebutuhan	Sintaks
List kosong	`r-empty="items"`
Atribut dinamis	`:href`, `:disabled`, `:class="{ active: x }"`
Fragment partial render	`r-fragment="nama"`
Pembungkus tanpa output	`<r-group>`
Komponen	`<x-product-card :product="p">` dari `partials/product-card.html`
Komponen modul lain	`<billing:plan-badge :user="user" />`
Props komponen	`<r-props product="Product" compact="bool = false" />`
Isi komponen	`<slot />`, `<slot name="actions" />`
Slot antar modul	`<r-slot name="admin.sidebar" />`

Bahasa ekspresi sengaja dibatasi

Ekspresi hanya mendukung akses properti, literal, perbandingan, `&&`, `||`, `!`, `??`, dan filter. Function call, assignment, dan side effect tidak didukung.

Alasan: logic yang tidak bisa ditulis di template akan dipindahkan ke loader TypeScript, yang memang punya type checking dan bisa di-test. Ekspresi yang terbatas juga bisa dievaluasi Rust tanpa menyentuh engine JS, sehingga render tidak menambah beban heap JS.

Kenapa interpolasi memakai `{{ }}`

Kurung kurawal tunggal bentrok dengan atribut client yang berisi object atau JSON, seperti `x-data="{ open: false }"` dan `hx-vals='{"id": 1}'`. Interpolasi juga tidak diproses di dalam `<script>` dan `<style>`.

Pipeline compile

1. Parse HTML dengan `html5ever` (parser HTML5 yang sesuai spesifikasi).
2. Resolve layout, komponen, slot, dan override modul.
3. Validasi: `r-else` tanpa `r-if`, komponen yang tidak ada, props yang salah, ekspresi tidak valid. Error dilaporkan dengan nama file dan nomor baris.
4. Compile menjadi daftar instruksi (tulis string statis, evaluasi ekspresi, loop, cabang). Bagian statis yang berurutan digabung menjadi satu string.

5. Simpan ke cache, dengan key berupa hash isi file.

Saat request, renderer hanya menjalankan instruksi dan menulis ke satu buffer output. Tidak ada parsing di jalur request.

Opsi kedua: TSX

Loader boleh mengembalikan JSX yang dirender di isolate. Opsi ini untuk halaman dengan logic tampilan yang rumit atau tim yang lebih suka gaya React. Konsekuensinya, render memakai CPU dan heap JS, sehingga tidak dijadikan default.

7. Partial rendering dan interaktivitas

Untuk update sebagian halaman, server mengirim fragment HTML dan browser hanya menukar bagian yang berubah. Satu template dan satu loader melayani render penuh maupun render sebagian, jadi tidak ada endpoint tambahan yang perlu ditulis.

Alur request fragment

1. Request pertama ke `/products` dirender penuh: layout dan semua fragment.
2. htmx mengirim `GET /products?q=sepatu` dengan header `HX-Request: true` dan `HX-Target: product-list`.
3. Rust menjalankan loader yang sama, lalu hanya merender elemen `r-fragment` yang `id`-nya cocok dengan target. Layout tidak dirender.
4. Response berisi beberapa KB HTML. htmx men-swap fragment tersebut dengan ekstensi `morph`.

Untuk action yang perlu meng-update beberapa bagian sekaligus (misalnya tombol dan badge keranjang), Rust merender fragment tambahan dan menandainya dengan `hx-swap-oob="true"` secara otomatis.

Kenapa transisinya smooth

- **DOM morphing (idiomorph), bukan `innerHTML`.** Hanya node yang berbeda yang diubah. Focus input, posisi scroll, media yang sedang diputar, dan animasi CSS yang sedang berjalan tidak ter-reset.
- **View Transitions API.** Swap dibungkus `document.startViewTransition`, jadi browser menganimasikan perubahan state. Elemen dengan `view-transition-name` akan bergeser dengan halus ke posisi barunya.

Lapisan interaktivitas

Kebutuhan	Solusi	Dikerjakan oleh
Halaman pertama	SSR penuh	Rust + loader TS
Filter, paging, form	Fragment + morph	Rust render, htmx swap
Animasi antar state	View Transitions + CSS	Browser
Dropdown, modal, tab	Alpine.js atau vanilla JS	Browser, tanpa request ke server
Widget sangat interaktif	Islands (Preact/Svelte/Solid)	Browser. Bundle dibuat Rust (swc/oxc) saat deploy
Realtime	SSE/WebSocket yang mengirim fragment HTML	Rust

Alasan memakai htmx, bukan client buatan sendiri: htmx dan idiomorph sudah matang dan terdokumentasi. Platform cukup memahami header-nya di sisi server. Client sendiri baru layak dibuat kalau nanti butuh integrasi yang lebih rapat, misalnya validasi form otomatis dari schema loader.

Realtime: SSE dan WebSocket

SSE dan WebSocket didukung sebagai fitur inti. **Koneksi dipegang oleh Rust, bukan oleh JS.** Isolate hanya dipanggil saat ada event (connect, pesan masuk, disconnect), sehingga ribuan koneksi yang sedang idle tidak memakai heap JS sama sekali. Biayanya hanya beberapa KB per koneksi di tokio.

	SSE	WebSocket
Arah	Server → browser	Dua arah
Cocok untuk	Notifikasi, feed, progress, live update fragment	Chat, kolaborasi, game, input berfrekuensi tinggi
Integrasi htmx	`hx-ext="sse"`, `sse-connect`, `sse-swap`	`hx-ext="ws"`, `ws-connect`, `ws-send`
Lewat proxy/CDN	Sangat mudah (HTTP biasa)	Butuh dukungan upgrade

Konvensi file:

```
// functions/ws/chat/[room].ts  ->  wss://host/ws/chat/:room
export const ws = {
  async open({ conn, params, auth, realtime }) {
    await auth.requireUser(conn.req);
    realtime.subscribe(conn, `room:${params.room}`);
  },
  async message({ conn, data, params, realtime, render }) {
    const html = await render("partials/chat-message.html", { msg: data,
user: conn.user });
    realtime.publish(`room:${params.room}`, html);     // fragment HTML ke
semua subscriber
  },
  close({ conn }) { /* opsional */ },
};
```

```
<!-- SSE: fragment otomatis di-swap oleh htmx -->
<div hx-ext="sse" sse-connect="/sse/orders" sse-swap="order-updated">…</div>
```

Pub/sub di Rust. `realtime.publish(topic, payload)` bisa dipanggil dari mana saja: handler WebSocket, action, service, atau event handler durable. Topic dikelola oleh broadcast channel tokio. Contohnya, setelah `billing.invoice_paid` di-commit, handler mem-publish fragment baru ke topic `user:<id>`, dan halaman user langsung ter-update lewat SSE.

Lebih dari satu instance server: pesan diteruskan antar instance lewat Postgres `LISTEN/NOTIFY`, jadi tidak perlu Redis di awal. Kalau volume pesan sudah melewati kemampuan NOTIFY, transport bisa diganti dengan NATS atau Redis tanpa mengubah API.

Keamanan dan keandalan:

- Autentikasi saat connect, memakai cookie session yang sama.
- Otorisasi per topic lewat hook `realtime.authorize`, misalnya user hanya boleh subscribe ke `user:<id-nya-sendiri>`.
- Heartbeat/ping, batas jumlah koneksi per user, dan batas ukuran pesan.
- Backpressure: kalau client terlalu lambat, buffer-nya dibatasi lalu koneksi ditutup, supaya memory server tidak membengkak.
- SSE mendukung `Last-Event-ID`, sehingga client yang reconnect bisa menerima event yang terlewat dari buffer singkat.

Tahap lanjut: realtime dari perubahan database (seperti Supabase Realtime) memakai logical replication Postgres, lalu dipetakan ke topic per tabel dengan filter RLS.

8. Sistem modul

Modul adalah mini app dengan struktur folder yang sama seperti app. Loader yang sama dipakai untuk keduanya, sehingga developer cukup memahami satu konsep. Karena isinya hanya TS, HTML, dan SQL, memasang modul tidak membutuhkan rebuild Rust.

```
modules/billing/
├── module.toml     manifest
├── migrations/     SQL, schema billing
├── pages/          halaman SSR
├── functions/      API
├── partials/       komponen yang bisa dipakai modul lain
├── public/         asset statis
├── services.ts     kontrak publik: fungsi untuk modul lain
└── hooks.ts        event handler, hook, middleware
```

Manifest

```
[module]
name = "billing"
version = "1.2.0"

[dependencies]
auth = "^1.0"

[config]
currency = { type = "string", default = "IDR" }
provider = { type = "enum", values = ["stripe", "midtrans", "xendit"] }

[permissions]
network = ["api.midtrans.com"]
db_schema = "billing"

[exports]
views = ["billing_public.user_plans"]

[[slots]]
name = "admin.sidebar"
template = "partials/admin-menu.html"
```

Komposisi di app

```
# renox.toml
[modules]
auth    = { version = "^1.0" }
"2fa"   = { version = "^1.0" }
billing = { version = "^1.2", mount = "/billing", config = { provider =
"midtrans" } }
admin   = { git = "https://github.com/org/renox-admin", tag = "v0.3", mount =
"/admin" }
blog    = { path = "../shared/blog", mount = "/blog" }
```

Saat startup, module resolver menjalankan langkah berikut:

1. Resolve versi semua modul (path, git, nanti registry), lalu tulis `renox.lock`.
2. Bangun graph dependency. Startup ditolak kalau ada siklus atau versi yang tidak cocok.
3. Jalankan migration per modul sesuai urutan topologis graph.
4. Gabungkan route dengan prefix `mount`. Startup ditolak kalau ada route yang bentrok.
5. Daftarkan services, event handler, hook, dan slot ke Module Broker.
6. Siapkan context JS per modul dari snapshot.

Override tanpa fork

Urutan pencarian template: `overrides/<modul>/...` → `modules/<modul>/...`. Hanya tampilan yang bisa ditimpa. Loader, services, dan migration tetap milik modul, sehingga bugfix dari upgrade modul tetap diterima.

Setiap override menyimpan hash file asli saat override dibuat. Kalau file asli berubah setelah upgrade, `renox dev` menampilkan peringatan beserta diff-nya.

Alasan override dibatasi ke template: kalau logic boleh di-override, setiap upgrade modul berisiko merusak app pemakai secara diam-diam. Untuk mengubah perilaku, gunakan hook atau event yang memang disediakan modul.

9. Komunikasi antar modul

Modul tidak pernah menyentuh kode atau tabel modul lain secara langsung. Semua komunikasi melewati Module Broker di Rust, yang memeriksa izin, meneruskan konteks, dan mengelola transaksi. Ada lima mekanisme, masing-masing untuk kebutuhan yang berbeda.

Mekanisme	Sifat	Butuh dependency	Dipakai saat
Service call	Sinkron, request/response	Ya	Pemanggil butuh jawaban sekarang
Event	Asinkron, fire-and-forget	Tidak	Mengumumkan bahwa sesuatu sudah terjadi
Hook	Sinkron, berurutan, bisa menghentikan alur	Tidak	Ikut menentukan apakah proses milik modul lain boleh lanjut
Public view	Read-only SQL	Ya	Join atau filter data lintas modul
Slot	Penyisipan template	Tidak	Menampilkan UI di halaman modul lain

Service call

```
// modules/billing/services.ts
export async function getActivePlan(ctx, userId: string): Promise<Plan |
null> { /* ... */ }

// pemanggil
import { billing } from "renox:modules";
const plan = await billing.getActivePlan(user.id);   // ctx diisi oleh broker
```

Langkah broker untuk setiap panggilan:

1. Pastikan pemanggil mendeklarasikan modul tujuan di `dependencies`. Kalau tidak, panggilan ditolak.
2. Serialisasi argumen ke nilai netral (serde). Object JS tidak pernah dibagi antar context.
3. Masuk ke context modul tujuan, dengan `ctx` milik modul tersebut: `db` yang dibatasi ke schema-nya, config-nya, serta user, session, dan transaksi yang sama dengan pemanggil.
4. Jalankan fungsi, serialisasi hasil atau error, lalu kembali ke context pemanggil.

Semua langkah terjadi di thread dan isolate yang sama, tanpa HTTP. Targetnya overhead di level mikrodetik, yang perlu dibuktikan lewat benchmark.

Saat `renox dev`, Rust membaca `services.ts` setiap modul dan meng-generate `renox-modules.d.ts`, sehingga panggilan antar modul punya autocomplete dan type checking.

Event

	In-process	Durable (default)
Waktu eksekusi	Langsung, di request yang sama	Setelah commit, oleh worker Rust
Kalau handler gagal	Request ikut gagal	Retry dengan exponential backoff, lalu dead-letter
Kalau server restart	Event hilang	Tetap terkirim
Jaminan	Tepat sekali, dalam request	At-least-once, handler harus idempotent (`e.id` untuk deduplikasi)

Durable event memakai transactional outbox: baris event ditulis ke `renox.outbox` di transaksi yang sama dengan perubahan data. Dengan begitu tidak ada event tanpa data, dan tidak ada data tanpa event. Worker Rust membaca outbox dengan `FOR UPDATE SKIP LOCKED` dan dibangunkan oleh `LISTEN/NOTIFY` supaya tidak perlu polling.

Hook

Modul pemilik proses menyediakan titik hook (`ctx.hooks.run("auth.after_login", ...)`). Modul lain mengisinya dengan `priority`. Handler dijalankan berurutan dan boleh mengembalikan `{ halt: true, response }` untuk menghentikan alur. Contohnya, modul 2FA mengalihkan user ke `/2fa/verify` setelah login berhasil, tanpa mengubah kode modul auth.

Public view

Modul boleh meng-export view di schema `<modul>_public`, yang menjadi kontrak baca publik. Modul lain yang mendeklarasikan dependency boleh membacanya (read-only) dan melakukan join. Tabel internal tetap privat, jadi modul pemilik bebas mengubah struktur tabelnya selama view tetap kompatibel.

Alasan: kalau data lintas modul hanya bisa diambil lewat service call, halaman list seperti admin akan terkena masalah N+1 query.

Konteks yang ikut di setiap panggilan

Konteks	Fungsi
`user`, `session`	Identitas yang sama di seluruh rantai panggilan

Konteks	Fungsi
`transaction`	Service call ikut transaksi pemanggil
`request_id`, trace	Satu request bisa dilacak lintas modul di log dan tracing
`deadline`	Timeout berlaku sampai ke panggilan terdalam

Transaksi lintas modul

```
return ctx.db.transaction(async () => {
  const order   = await orders.create(cart);
  const invoice = await billing.createInvoice(user.id, order);
  await inventory.reserve(order.items);   // gagal → order dan invoice ikut
di-rollback
});
```

Broker memakai satu koneksi Postgres per transaksi dan meneruskannya ke setiap service call. Ini keuntungan utama arsitektur in-process dibanding microservices, yang membutuhkan saga atau distributed transaction untuk hasil yang sama.

Aturan yang ditegakkan

1. Service call dan public view wajib dependency eksplisit.
2. Dependency melingkar ditolak saat startup. Kalau dua modul saling butuh, salah satu arah diganti dengan event.
3. `db` milik modul hanya bisa menulis ke schema modul itu sendiri.
4. Nama event, signature service, nama hook, nama slot, dan view adalah public API modul, sehingga di-versioning dengan semver.

10. Data, keamanan, dan sandbox

PostgreSQL adalah satu-satunya penyimpanan state. Setiap batasan keamanan ditegakkan oleh Rust, bukan oleh kode JS, sehingga modul yang ditulis dengan buruk tetap tidak bisa melewati batas izinnya.

Data

- **Schema per modul:** `auth`, `billing`, `twofa`, dan seterusnya. Schema `renox` dipakai platform untuk menyimpan riwayat migration, outbox, dan session.
- **Migration** disimpan per modul di `renox.migrations (module, version, checksum, applied_at)`. Checksum mencegah file migration lama diubah diam-diam.

- **Akses DB dari JS** lewat op Rust (`sqlx` / `tokio-postgres`) dengan connection pool bersama. Query selalu memakai parameter (`$1`), tidak ada interpolasi string.
- **Role Postgres per modul**, dengan `GRANT` hanya ke schema miliknya dan `SELECT` ke view `*_public` dari dependency-nya. Isolasi ditegakkan oleh database, bukan hanya oleh broker.

Tingkat isolasi modul

Mode	Cara kerja	Biaya panggilan	Keamanan	Untuk
`shared`	Semua modul di-bundle ke satu context	Panggilan fungsi JS biasa	Konvensi saja	Modul milik satu tim
`context` (default)	Satu context per modul dalam isolate yang sama	Mikrodetik, plus serialisasi	Global terpisah, izin ditegakkan Rust	Kebanyakan modul
`isolate`	Isolate terpisah dengan heap limit sendiri	Lebih mahal, lintas isolate	Crash, infinite loop, dan memory leak terisolasi	Modul pihak ketiga

Batasan runtime

Batasan	Mekanisme
CPU time per request	Watchdog thread memanggil `terminate_execution` V8 kalau melewati batas
Memory	Heap limit per isolate dan near-heap-limit callback
Wall-clock	`deadline` di context, query dibatalkan saat lewat batas
Network	Allowlist host di `module.toml` , ditegakkan op `fetch`
Filesystem	Tidak ada akses langsung. Akses file hanya lewat API storage
Env/secret	Hanya secret yang dideklarasikan modul yang disuntikkan ke context

State global di context

Context modul dipakai ulang lintas request, sehingga global variable terbagi antar user. Data per request wajib lewat `ctx`, dan global hanya boleh dipakai untuk cache yang memang dimaksudkan untuk dibagi. Mode `renox dev` memberi peringatan kalau sebuah handler menulis ke global scope.

Web

- Template melakukan auto-escape secara default. HTML mentah hanya lewat `r-html` yang eksplisit.
- Proteksi CSRF untuk semua action non-GET, termasuk request htmx (token dikirim lewat header).
- Session memakai cookie `HttpOnly`, `Secure`, dan `SameSite=Lax`.
- Header keamanan bawaan: CSP, `X-Content-Type-Options`, dan `Referrer-Policy`.

11. Otorisasi: RBAC, ABAC, dan relasi

Otorisasi user ditangani oleh satu policy engine di Rust yang dipakai oleh semua modul. RBAC menjadi dasar, ABAC dan relasi (ReBAC) dipakai untuk aturan detail, dan Row Level Security Postgres menjadi pengaman terakhir untuk isolasi data. Semua penegakan dilakukan Rust, sehingga modul yang ditulis dengan buruk tidak bisa melewatinya.

Dua jenis permission

	Permission modul (capability)	Permission user (otorisasi)
Pertanyaan	Kode modul ini boleh melakukan apa?	User ini boleh melakukan apa?
Contoh	Billing boleh akses `api.midtrans.com` dan schema `billing`	Budi boleh me-refund invoice milik organisasinya
Ditentukan oleh	`module.toml`, disetujui saat install	Role dan policy, diatur admin saat runtime
Ditegakkan oleh	Sandbox Rust dan role Postgres (bagian 10)	Policy engine dan RLS (bagian ini)

Model

Model	Dipakai untuk	Contoh
RBAC	Mayoritas izin, mudah dikelola lewat UI admin	Role `finance` boleh membaca dan me-refund invoice
ABAC	Aturan berbasis atribut user, resource, atau konteks	Refund di atas Rp 10 juta hanya oleh `finance_manager`, aksi sensitif wajib 2FA
ReBAC	Kepemilikan dan keanggotaan	Pemilik dokumen, anggota organisasi, admin tim

Keputusan: Cedar sebagai policy engine

Policy ditulis dengan Cedar dan dievaluasi oleh crate Rust `cedar-policy`, dibungkus trait `PolicyEngine` supaya bisa diganti.

Alasan: Cedar berjalan native di Rust tanpa proses terpisah. Policy berupa file teks yang di-load saat runtime, jadi bisa diubah tanpa rebuild. RBAC, ABAC, dan relasi tercakup dalam satu bahasa, aturan `forbid` selalu menang atas `permit`, dan policy bisa divalidasi terhadap schema sebelum deploy.

Alternatif yang ditolak: OPA/Rego (ditulis dengan Go, perlu sidecar), Casbin (ABAC kurang ekspresif), dan predicate TypeScript buatan sendiri (tidak bisa dianalisis, berjalan di heap JS).

Deklarasi oleh modul

```
# modules/billing/module.toml
[authz]
resources = ["Invoice"]
actions = ["invoice.read", "invoice.create", "invoice.refund"]
sensitive = ["invoice.refund"]          # wajib 2FA dan dicatat di audit log

[authz.default_roles]                   # saran role, bisa diubah admin app
finance = ["invoice.read", "invoice.refund"]
viewer  = ["invoice.read"]
```

Modul menyediakan policy default di `modules/<modul>/policies/*.cedar`. App bisa menambah policy di `policies/` dan menimpa policy modul lewat `overrides/<modul>/policies/`.

```
// RBAC: finance boleh membaca dan me-refund invoice di organisasinya sendiri
permit (
  principal in Role::"finance",
  action in [Action::"invoice.read", Action::"invoice.refund"],
  resource is Billing::Invoice
) when { resource.org == principal.org };

// ABAC: refund besar hanya oleh finance_manager
forbid (principal, action == Action::"invoice.refund", resource)
when { resource.amount > 10000000 } unless { principal in
Role::"finance_manager" };

// Aksi sensitif wajib sesi yang sudah lolos 2FA
forbid (principal, action in Action::"sensitive", resource)
unless { context.mfa_verified };
```

Assignment role disimpan di database dan dibatasi per scope:

```
renox.role_assignments (user_id, role, scope_type, scope_id)
-- Budi = finance @ org:acme, viewer @ org:globex
```

Titik penegakan

Lapisan	Cara	Fungsi
Route	`export const guard = { action: "invoice.read" }` di loader/function	Menolak sebelum handler berjalan
Aksi pada resource	`await ctx.authz.require("invoice.refund", invoice)`	Pemeriksaan ABAC dengan atribut resource yang sebenarnya
Data (list)	Row Level Security Postgres	Filter baris per tenant/pemilik di database
UI	`r-can="invoice.refund"`	Hanya menyembunyikan elemen. **Bukan pengaman**
Realtime	Hook `realtime.authorize` per topic	Mencegah subscribe ke topic orang lain

```
// modules/billing/pages/invoices/[id].ts
export const guard = { action: "invoice.read" };

export const actions = {
  async refund({ ctx, params }) {
    const invoice = await getInvoice(ctx, params.id);
    await ctx.authz.require("invoice.refund", invoice);   // 403 kalau
ditolak
    // ...
  },
};
```

Policy engine dan RLS saling melengkapi

Policy engine menjawab "bolehkah user X melakukan aksi Y pada resource Z?" dengan cepat. Untuk halaman daftar, memeriksa ribuan baris satu per satu terlalu lambat, sehingga filter tenant dan kepemilikan diserahkan ke RLS.

```
alter table billing.invoices enable row level security;
create policy tenant_isolation on billing.invoices
  using (org_id = current_setting('renox.org_id')::uuid);
```

Di awal setiap transaksi request, Rust menjalankan `set_config('renox.user_id', ..., true)` dan `set_config('renox.org_id', ..., true)`. Kalau kode modul lupa menambahkan filter `org_id`, database tetap menolak baris milik tenant lain.

Alasan dua lapisan: RLS memberi defense in depth untuk isolasi data, sedangkan Cedar memberi aturan bisnis yang detail. Mengandalkan salah satunya saja berarti antara lambat untuk list, atau tidak cukup ekspresif untuk aturan bisnis.

Konteks lintas modul

- Identitas user ikut di setiap service call. **Setiap modul menegakkan policy untuk resource miliknya sendiri.** Billing yang memeriksa izin invoice, bukan modul pemanggil.
- Event handler, cron, dan job berjalan sebagai principal `System::"<nama-modul>"` dengan izin eksplisit, bukan superuser.
- Impersonasi admin dicatat di context dan audit log, termasuk siapa yang melakukan impersonasi.

Performa

- Role, keanggotaan, dan atribut user di-load sekali per request, lalu di-cache di Rust.
- Cache di-invalidate lewat `LISTEN/NOTIFY` saat role atau policy berubah.

- Policy di-parse dan divalidasi terhadap schema saat deploy, bukan saat request.

Audit dan pengujian

- Keputusan untuk action `sensitive` dicatat di tabel append-only: siapa, aksi, resource, hasil, dan policy penentu.
- `renox authz explain --user budi --action invoice.refund --resource inv_123` menampilkan policy yang mengizinkan atau menolak.
- `renox test` menjalankan file kasus uji policy (principal, action, resource, hasil yang diharapkan).

Lapisan keamanan terkait

Area	Desain
Autentikasi	Modul `auth`: argon2id, OAuth/OIDC, magic link. Session cookie untuk browser, API key atau JWT untuk mesin
2FA	Modul `2fa`: TOTP dan WebAuthn/passkey, lewat hook `auth.after_login` dan atribut `context.mfa_verified`
Rate limiting	Per IP, per user, dan per route di gateway, dengan aturan ketat untuk login dan reset password
Validasi input	Schema validation di action dan function sebelum handler berjalan
Secret	Terenkripsi, hanya disuntikkan ke modul yang mendeklarasikannya
Supply chain modul	Checksum di `renox.lock`, persetujuan permission saat install, diff permission saat upgrade
Audit log	Append-only untuk login, perubahan role, dan aksi sensitif

12. Developer experience

Semua alat developer ada di binary yang sama dengan server. Tidak ada toolchain tambahan selain editor.

Perintah	Fungsi
`renox new <nama>`	Membuat project dari template
`renox dev`	Server lokal dengan hot reload, error overlay, dan generate type
`renox add <modul>`	Menambah modul ke `renox.toml` dan memperbarui `renox.lock`
`renox migrate`	Menjalankan migration semua modul sesuai urutan dependency
`renox check`	Type check TS, validasi template, dan deteksi konflik route
`renox test`	Menjalankan test TS di runtime yang sama dengan produksi
`renox build`	Memaketkan app menjadi artifact yang immutable (bundle + template ter-compile + hash)
`renox deploy`	Mengirim artifact ke server, lalu melakukan atomic swap

Dev loop

1. File watcher (`notify`) mendeteksi perubahan.
2. Hanya file yang berubah yang diproses: TS di-transpile dengan swc (`deno_ast`), template di-compile ulang.
3. Context modul yang terdampak dibuat ulang. Isolate dan koneksi DB tetap hidup.
4. Browser di-refresh lewat SSE. Kalau yang berubah hanya template, cukup fragment yang di-swap.

Target dari save sampai browser ter-update: di bawah 300 ms untuk perubahan satu file.

Type safety

`renox dev` meng-generate `.renox/types/` yang berisi type `LoaderContext` , `params` per route (dari nama file), services setiap modul, dan payload event. Type checking memakai `tsc` atau tsgo di editor. Runtime sendiri tidak melakukan type check, supaya hot reload tetap cepat.

Deploy tanpa downtime

1. `renox build` menghasilkan artifact dengan hash isi.
2. Server memvalidasi artifact (route, template, dependency modul), lalu menjalankan migration.
3. Snapshot context baru disiapkan di samping yang lama.
4. Router di-swap secara atomic. Request yang sedang berjalan diselesaikan dengan versi lama.
5. Rollback cukup dengan swap ke hash sebelumnya, selama migration-nya backward compatible.

Observability

Tracing OpenTelemetry dengan span per request, per service call antar modul, per query, dan per render. Metric per route dan per modul: latency, error rate, CPU time JS, dan heap. Log terstruktur selalu menyertakan `request_id` dan nama modul.

13. Risiko dan mitigasi

Risiko terbesar bukan di Rust, melainkan di manajemen isolate, kompatibilitas library JS, dan disiplin kontrak antar modul.

Risiko	Dampak	Mitigasi
Scope terlalu besar (meniru seluruh Supabase)	Proyek tidak pernah sampai ke versi yang bisa dipakai	Roadmap bertahap. Fase awal hanya pages, functions, template, dan modul
Library npm butuh Node API	Developer tidak bisa memakai library favoritnya	Pakai ulang polyfill `deno_node` secara bertahap dan dokumentasikan daftar kompatibilitas. Prioritaskan library berbasis Web API
Global state bocor antar request	Data user tercampur	Data per request wajib lewat `ctx`, ada peringatan di mode dev, dan mode `isolate` untuk modul yang tidak dipercaya
Ukuran binary dan waktu build V8	CI lambat, binary sekitar 30–60 MB	Pakai prebuilt `rusty_v8`. Build Rust hanya dilakukan saat platform berubah, bukan saat aplikasi berubah

Risiko	Dampak	Mitigasi
Memory V8 lebih besar dari target	Kurang cocok untuk VPS kecil	Abstraksi `JsEngine` memungkinkan pindah ke QuickJS, plus heap limit dan pembatasan jumlah worker
Coupling antar modul lewat tabel	Upgrade modul merusak modul lain	Role Postgres per modul memblokir akses tabel. Data lintas modul hanya lewat services, events, dan view
Kontrak modul berubah tanpa pemberitahuan	App pemakai rusak setelah upgrade	Semver, type yang di-generate, dan `renox check` mendeteksi kontrak yang hilang
Override template tertinggal versi	Tampilan rusak setelah upgrade	Hash file asli disimpan dan diff ditampilkan saat upgrade
Membangun template engine sendiri	Bug dan fitur yang kurang	Bahasa ekspresi minimal, test suite berbasis snapshot, dan fuzzing parser
Outbox membengkak	Disk dan latency DB naik	Partisi atau pembersihan berkala event yang sudah selesai, dan tabel dead-letter
Realtime pada skala besar	NOTIFY menjadi bottleneck	Transport pub/sub dibuat pluggable (NATS/Redis)

14. Roadmap dan pertanyaan terbuka

Setiap fase menghasilkan sesuatu yang bisa dijalankan, dan fase berikutnya baru dimulai setelah kriteria selesai terpenuhi. Urutannya mengikuti ketergantungan: modul bergantung pada runtime, dan komunikasi antar modul bergantung pada modul.

1. **F0: Fondasi.** Cargo workspace, gateway axum, static hosting dari `public/`, dan config `renox.toml`.
   - Selesai kalau: file statis ter-serve dengan ETag, kompresi, dan cache header, serta lolos benchmark baseline.
2. **F1: Function runtime.** `deno_core`, abstraksi `JsEngine`, worker pool, transpile TS dengan swc, `functions/`, op `fetch` dan `db`.
   - Selesai kalau: `functions/api/*.ts` berjalan, hot reload < 300 ms, dan CPU/heap limit berfungsi.

3. **F2: SSR dan template.** Parser dan compiler template atribut, routing `pages/`, loader, layout, komponen, dan actions.
   - Selesai kalau: contoh toko (list + detail + form) berjalan penuh dengan SSR.
4. **F3: Partial rendering dan realtime.** `r-fragment`, integrasi header htmx, OOB swap, SSE, WebSocket, dan pub/sub.
   - Selesai kalau: search live, keranjang yang ter-update, dan chat room berjalan tanpa reload halaman.
5. **F4: Sistem modul.** Manifest, resolver dan lockfile, mount, migration per schema, role Postgres per modul, dan overrides.
   - Selesai kalau: modul `auth` dan `blog` terpasang dari path dan git.
6. **F5: Komunikasi antar modul.** Module Broker, service call, type yang di-generate, event in-process, hook, slot, public view, serta policy engine Cedar, RLS per tenant, dan ctx.authz.
   - Selesai kalau: modul `2fa` menyisip ke login `auth` tanpa mengubah kode `auth`, dan refund invoice ditolak untuk user tanpa role yang sesuai atau tanpa 2FA.
7. **F6: Keandalan.** Durable event dengan outbox, deploy atomic, rollback, OpenTelemetry, dan mode isolasi `isolate`.
   - Selesai kalau: event tetap terkirim setelah server di-kill di tengah proses, dan deploy tanpa request yang gagal.
8. **F7: Ekosistem.** Modul resmi (`admin`, `billing`, `storage`), islands, TSX opsional, dan realtime dari perubahan DB.

Pertanyaan terbuka

- **Engine default:** V8 atau QuickJS? Putuskan setelah benchmark di F1 dengan beban nyata (render list 100 item, 10 query per request).
- **Target perangkat minimum:** VPS 512 MB atau 1 GB? Jawabannya menentukan jumlah worker default dan pilihan engine.
- **Multi-tenant:** apakah Renox akan di-host untuk banyak pelanggan (seperti Supabase cloud) atau self-host per aplikasi? Multi-tenant mengharuskan mode `isolate` dan quota sejak awal.
- **Auth:** dibangun sendiri sebagai modul, atau kompatibel dengan GoTrue dari Supabase?
- **REST API otomatis dari schema** (seperti PostgREST): dibutuhkan di fase awal, atau cukup lewat functions?
- **Lisensi dan distribusi modul:** perlu registry sendiri, atau cukup git tag?